Make Your ITEM (with number) · 2026 유니티 게임잼 기획 정리

> 숫자 함수와 영문 변수로 제작 프로그램을 작성하고, 3D 대장장이가 만든 장비를 자유롭게 팔아 공방을 성장시키는 게임.

주제 행운의 숫자 · **게임 이름** Make Your ITEM (with number) · **플랫폼** Unity 6, 3D 싱글 플레이 · **조작** 마우스 클릭과 키보드 입력

대상 Unity 게임 개발자와 코딩을 즐기는 플레이어. **제작 계획** 기능 구현에 ChatGPT, 3D 모델 제작에 Meshy 활용. 함수와 재료를 숫자로 표현하는 자체 언어를 만들고, 코딩·아이템 제작·타이쿤의 성장을 연결한다. 시스템 참고작은 《농부는 대체되었다》.

1. 현재 방향과 핵심 규칙
2. 플레이 흐름과 화면
3. 숫자 도면 언어
    도면 함수·표기 레퍼런스
4. 컴파일러, 가상 머신과 오류 처리
5. 판매와 발견
6. 행운의 숫자, 상점과 해금 후보
7. 게임잼 구현 우선순위
8. 보류 중인 아이디어 · 4~9번 재료와 특수 가공
    용뼈 정제 예시
    재료 조합을 발견하는 예시
9. 확정 예약어와 인게임 적용 프롬프트
    인게임 숫자 예약어 및 구문 강조 적용 프롬프트

1. 현재 방향과 핵심 규칙

플레이어는 기본 공정을 짧게 익힌 뒤 원하는 장비를 만들고 자유 판매한다. 제작법은 재료 설명, 코드 툴팁, 공정 결과, 판매 영수증을 통해 스스로 발견한다. 현재의 기본 플레이 흐름은 자유 제작과 판매다.

> 같은 재료라도 어느 공정에 어떤 상태에서 쓰느냐에 따라 결과가 달라진다. 판매가는 완성품이 실제로 얻은 성질에 따라 올라가고, 이익은 그 가격에서 제작비를 뺀 금액이다.

예시 뜨거운 철 검날을 물로 식히면 담금질로 경도가 오른다. 식은 검날에 물을 쓰면 물값이 들고 경도 변화는 없다. 나무는 부품 장착에서 손잡이, 화로에서 연료가 될 수 있다. 플레이어는 코드의 순서, 조건, 재료를 바꾸며 어떤 변경이 수익으로 이어지는지 알아간다.

2. 플레이 흐름과 화면

1. **튜토리얼** 철을 올리고 가열·타격·냉각해 검날을 판매하는 기본 루프를 한 번 경험한다.

2. **코딩과 시험 실행** 숫자 도면을 작성하고, 재료를 소비하지 않는 시험에서 줄별 결과와 예상 비용을 확인한다.
3. **실제 제작** 컴파일한 도면을 대장장이가 실행한다. 공정마다 제작 시간이 걸리고 3D 대장간에 행동과 상태 변화가 드러난다.
4. **검사와 자유 판매** 완성품의 성질과 가격 내역을 확인한 뒤 판매한다. 완성 검 외에 판매 조건을 갖춘 검날도 팔 수 있다.
5. **상점과 성장** 수익으로 공정 가속, 보호, 저주, 자동판매, 특수 재료 등을 해금한다. 새 기능을 도면에 적용해 이익, 생산 속도, 재료 효율을 개선한다.
6. **반복과 엔딩** 제작과 판매를 반복해 공방을 성장시킨다. 자유 판매 중심 엔딩 조건은 미정이다.

화면	역할
코드 에디터	사각형 코드 입력창. 상하좌우 가장자리를 드래그해 크기를 조절한다. 상단에 화살표 실행 버튼을 배치한다. 숫자 위에 커서를 올리면 문맥에 맞는 뜻과 재료 비용을 보여 준다.
콘솔과 실행 기록	컴파일 오류, 실행 오류, 경고를 구분한다. 실행한 줄을 강조하고 당시 인자·온도·경도 등의 값을 표시한다. 줄 단위 실행은 확장 후보로 둔다.
3D 대장간	작업대, 화로, 모루, 냉각 도구를 통해 대장장이의 행동과 공정 순서를 보여 준다. 증기와 수치 변화 등으로 공정 효과를 전달한다.
재료 진열대·상점	재료 번호, 용도, 비용과 구매 가능한 공방 기능을 보여 준다.
검사·판매 화면	기본 가격, 경도·부품 등으로 얻은 보너스, 제작비, 순이익을 영수증에 나눠 표시한다.

3. 숫자 도면 언어

소스는 영문 변수명, `0`~`9`로 구성한 숫자, 정해진 특수문자, 공백과 줄바꿈으로 작성한다. 변수는 `Material`이나 `Blade`처럼 선언과 참조에서 같은 이름을 그대로 쓴다. 괄호 앞 숫자는 함수 번호이며, 인자로 전달한 숫자는 함수의 인자 위치에 따라 재료 번호나 온도 등의 수치로 해석한다. 숫자는 여러 자리로 조합할 수 있다. Python처럼 들여쓰기로 조건 블록을 읽는 방향이다.

표현	뜻
`7`	정수 리터럴. 함수에서 재료 번호를 받는 자리에 전달하면 해당 재료 번호로 해석한다.
`Material` / `Material = 1`	`Material` 변수 참조 / `Material` 변수에 정수 `1`을 저장.
`1(...)`	번호가 `1`인 함수 호출.
`Blade = 1(Material)`	`Material`의 값으로 재료를 작업대에 올리고 반환된 작업물 참조를 `Blade`에 저장한다. 값이 `1`이면 철을 올린다.
`+`, `>=`, `? 조건:`	산술, 비교, 조건문 예시. 조건이 참일 때 들여쓴 블록을 실행한다.
`!Blade`	`Blade`의 작업물이 판매 가능한 상태인지 검사하고 완성품으로 반환한다.

초기 재료 번호 제안

번호	재료	공방에서의 사용 예시
`0`	공기	냉각 함수에 전달해 공기로 식힌다.
`1`	철	작업대에 올려 검날의 바탕으로 쓴다.
`2`	나무	손잡이 부품이나 연료로 활용한다.

번호	재료	공방에서의 사용 예시
`3`	물	냉각에 쓴다. 담금질 효과는 사용 직전의 상태에 따라 달라진다.
`4`		
`5`		
`6`		
`7`	별철	검날의 열 단계가 **정확히** `7` **일 때** 냉각을 시작하면 별무늬가 새겨집니다. `6`이나 `8`에서 식히면 일반 별철 검날이 됩니다.
`8`		
`9`		

기본 공방 함수 초안

호출	공정
`1(재료 번호)`	재료를 작업대에 올리고 작업물 참조를 돌려준다.
`2(작업물, 온도)`	작업물을 목표 온도까지 가열한다.
`3(작업물)`	가열된 철을 타격해 검날의 형태를 잡는다.
`4(작업물, 냉각 재료)`	공기 또는 물을 사용해 작업물을 식힌다.
`5(작업물, 부품 재료)`	형태가 잡히고 식은 검날에 손잡이 등의 부품을 장착한다.

도면 함수·표기 레퍼런스

상태 열은 현재 프로토타입의 지원 범위와 확장 계획을 구분합니다. 현재 재료 올리기는 철, 냉각은 공기를 지원합니다. 물 냉각과 손잡이 장착은 기획 단계입니다.

함수·표기	하는 일	반환값	비용	상태	설명
`1(material)`	재료를 올려 작업물 생성	작업물 참조	철 1개 + 10G	현재 지원, 철만	재료 올리기
`2(piece, target)`	목표 열 단계로 설정	없음	호출당 2G	현재 지원	가열
`3(piece)`	가열한 작업물을 검날로 성형	없음	0G	현재 지원	타격
`4(piece, coolant)`	작업물을 냉각	없음	공기 0G	현재 지원, 공기만	냉각
`!piece`	완성품 검사 및 도면 종료	판매 가능한 완성품 등록	추가 0G	현재 지원, 함수가 아닌 문장	완성품 판정
`5(piece, material)`	손잡이 등 부품 장착	미확정	미확정	공정 기획만 존재	장착 후보
`heat(piece)`	현재 열 단계 조회	정수 0~9	0G	확장 계획	열 조회
`abs(n)`	절댓값	정수	0G	확장 계획	절댓값
`len(values)`	원소 개수	정수	0G	확장 계획	길이
`min(values)`	최솟값	정수	0G	확장 계획	최솟값
`max(values)`	최댓값	정수	0G	확장 계획	최댓값
`random(low, high)`	양 경계를 포함해 정수 추첨	정수	0G	확장 계획	난수

함수·표기	하는 일	반환값	비용	상태	설명
`set()`	빈 정수 집합 생성	집합	0G	확장 계획	빈 집합
`이름(인자...)`	`def`로 작성한 사용자 함수 호출	`return`에 따라 결정	내부 공정의 합계	확장 계획	사용자 함수

`IronBlade.num` **· 기본 철 검날 도면**

```
Material = 1
Blade = 1(Material)
2(Blade, 7)
3(Blade)
4(Blade, 0)
!Blade
```

`4(Blade, 0)`의 `0`을 `3`으로 바꾸면 물 냉각을 시도한다. 작업물이 식은 뒤 `!Blade` 앞에 `5(Blade, 2)`를 추가하면 나무 손잡이를 단다. 에디터에서 `4(Blade, 3)`의 마지막 `3`에 커서를 올리면 '냉각 재료 · 물'이 표시된다. `2(Blade, 7)`의 마지막 `7`에는 '목표 온도 · 7'이 표시된다.

4. 컴파일러, 가상 머신과 오류 처리

소스 → 토큰화 → 구문 트리 → 함수 인자·변수·타입 검사 → 바이트코드 → 공방 가상 머신 → 제작 상태와 완성품 판정. 조건문은 바이트코드의 분기 명령으로 처리한다. 가상 머신은 공방 함수에 도달하면 작업물 상태를 검사하고 실제 제작에서는 3D 대장장이의 작업을 실행한다. 시험 실행과 실제 제작은 동일한 공정 규칙을 사용한다.

상황	판정	플레이어 피드백
`5()`처럼 작업물과 부품 인자를 생략함	컴파일 오류. 제작 시작 전 정지.	오류 줄과 필요한 인자 두 개를 표시.
`5(Blade, 2)`를 호출했지만 `Blade`를 선언한 적이 없음	컴파일 오류. 제작 시작 전 정지.	정의되지 않은 변수 `Blade`를 표시하고, `Blade = 1(1)`로 철을 작업대에 올려 저장하는 방법을 안내한다.
철괴를 올린 직후 손잡이를 장착함	실행 오류. 해당 공정 직전에 정지.	현재 상태가 철괴이며 형태를 잡고 식혀야 한다고 표시.
식은 검날에 물을 사용함	효과가 없는 공정. 경고 후 계속.	예시로 냉각 직전 온도 `4`, 담금질 기준 `6`, 물값 `2G`, 경도 변화 없음 표시.
`!Blade`로 미가공 철괴를 장비로 반환함	완성품 검사 실패. 판매 단계 진입 불가.	현재 상태와 필요한 공정을 표시.

비용과 상태 처리 제안 시험 실행은 실제 재고나 비용을 차감하지 않는다. 실제 제작이 실행 오류로 멈추면 이전 공정에서 쓴 재료와 연료는 비용에 남는다. 실패한 공정의 재료는 사용 전에 검사해 차감하지 않는다. 작업대에는 마지막으로 성공한 공정까지의 작업물이 남는다. 남은 작업물의 재개·회수·폐기 방식은 결정이 필요하다.

5. 판매와 발견

아래 가격과 시간은 경제 구조를 보여 주기 위한 예시다. 첫 판매 뒤 물의 툴팁과 영수증을 비교하면 냉각 재료 하나를 바꿀 이유가 생긴다. 경도 보너스나 손잡이 보너스를 판매가에 별도 항목으로 표시한다.

완성품	제작비	판매가	개당 이익	제작 시간
공기로 식힌 철 검날	12G	28G	16G	4초
물로 담금질한 철 검날	14G	47G	33G	7초
나무 손잡이가 붙은 담금질 검	19G	69G	50G	12초

검날에 물을 쓰는 데 2G가 추가되고 판매가가 19G 오른다면 개당 이익은 17G 늘어난다. 이 예시에서는 완성검의 개당 이익이 가장 높고 담금질 검날의 시간당 이익이 가장 높다. 공정 가속으로 손잡이 장착 시간이 줄면 생산 전략도 달라질 수 있다.

플레이어는 재료의 설명에서 가능성을 읽고, 3D 공정과 검사 수치로 결과를 확인하고, 판매 영수증으로 이익을 비교한다. 온도 읽기 함수와 조건문을 도입하면 검날이 뜨거울 때만 물을 쓰도록 해 재료비를 아낄 수 있다.

6. 행운의 숫자, 상점과 해금 후보

후보	논의한 효과와 미정 사항
경도 `7` · 행운의 날	경도가 정확히 `7`에 도달하면 숨은 특성을 발견한다. 발견 후 도감에 조건을 기록해 플레이어가 도면으로 재현하는 방향. 판매 보너스는 미정.
`777`	초기 제안은 `7`의 연속 사용에 7배 보상. 현재 방향에서는 입력 횟수보다 실제 공정 결과나 완성품의 성질과 연동하는 안을 검토한다. 발동 조건, 배율, 반복 적용 기준은 미정.
`666`	저주가 깃들어 판매가 10배를 노릴 수 있으나 일정 확률로 아이템이 파괴된다. `666(Blade)`를 공방 함수로 쓰는 안. 파괴율 25%와 보호 비용은 밸런스 예시.
`@`	저주의 아이템 파괴를 막는 보호 기능. `@666(Blade)` 표기가 후보이며 사용 비용과 정확한 문법은 미정.
`>` · `>>` · `>>>` · `>>>>`	제작 속도 2배·3배·4배·5배에 해당하는 가속 단계 초안. 최종 표기와 적용 공정은 미정.
자동판매 · `404`	자동판매는 완성품을 판매하도록 해금하는 기능. `404`는 실패한 제작의 재료를 일부 회수하는 별도 아이디어. 두 기능의 발동 조건은 미정.
언어 확장	출력·반복·괄호·예약어 해금, `8.123` 압축 표기 등을 초기 후보로 논의했다. 기본 문법을 제공하고 상점에서 공방 기능을 해금하는 안도 있으며, 최종 해금 범위는 미정.

특수 기능은 해금한 뒤에 사용할 수 있게 한다. 문법이 맞지만 해금되지 않은 함수를 호출했을 때는 상점에서 필요한 기능을 안내하는 방식이 후보이다.

7. 게임잼 구현 우선순위

1. **자유 판매 한 바퀴** 공기·철·나무·물 번호와 가열·타격·냉각을 연결해 철 검날을 만들고 판다.
2. **숫자 컴파일러와 에디터** 변수, 정수, 숫자 함수, 반환, 공정 상태 검사, 줄별 오류 메시지를 구현한다. 초기 검증은 변수 없는 장착, 철괴에 장착, 식은 검날의 물 냉각, 정상 제작을 포함한다.
3. **발견과 수익 차이** 공기 냉각, 뜨거운 상태에서 물 냉각, 식은 뒤 물 사용, 나무 손잡이를 비교한다. 툴팁, 검사 수치, 영수증으로 차이를 전달한다.
4. **공방 성장과 자동화** 첫 수익으로 가속 또는 자동판매를 해금해 도면과 수익이 어떻게 변하는지 확인한다. 온도 읽기와 조건문으로 낭비를 줄이는 플레이를 추가한다.
5. **행운과 엔딩** 경도 `7` 발견을 먼저 시험한다. `777`, `666`, `@`의 범위와 자유 판매 중심 엔딩 조건을 정한다.

8. 보류 중인 아이디어 · 4~9번 재료와 특수 가공

상태 · 보류. 아래 내용은 이후 검토할 재료·가공 제안이다. 숫자 조건과 수치는 기획 예시다.

재료 번호 `0`~`3`은 각각 공기, 철, 나무, 물이다. `4`~`9`는 플레이어가 투입량 계산, 공정 순서, 반복, 상태 확인, 두 값의 비교, 복합 조건을 각각 활용하도록 설계한 후속 재료 후보다.

번호 · 재료	가공 규칙	플레이어가 코드로 할 일	결과
`4` 석탄	화로에 넣는 연료다. 석탄 1개당 화로의 열 단계가 `2` 오른다. 목표보다 `2` 단계 이상 높아지면 과열로 표면 품질이 떨어진다.	현재 열을 읽고 필요한 개수만 투입한다. 현재 `3`, 목표 `7`이면 `2개`를 사용한다.	가열 시간과 연료 소비를 줄여 시간당 이익을 높일 수 있다.
`5` 은	식힌 검날을 연마한 뒤 은을 입혀야 도금이 고르게 붙는다. 연마 전에 도금하면 은을 소비하고 얼룩이 남는다.	검날의 연마 상태를 확인하고 필요할 때만 연마한다.	은빛 장식으로 판매가를 높인다.
`6` 마력 수정	식은 손잡이에 수정을 끼운 뒤 약한 마력을 `3회` 주입한다. 강하게 한 번 주입하면 수정에 금이 간다.	반복문으로 약하게 주입한다. 이미 충전된 수정은 남은 횟수만 주입한다.	마력 전도 특성을 얻는다. 용뼈 손잡이에 장착하면 용의 맥동을 추가로 얻는다.
`7` 별철	검날의 열 단계가 정확히 `7`일 때 냉각을 시작하면 별무늬가 새겨진다. `6`이나 `8`에서 냉각하면 일반 별철 검날이 된다.	냉각 직전의 열을 읽고, `7`보다 낮으면 가열하고 높으면 기다린다.	별무늬로 판매가를 높인다.
`8` 서리수	검날과 서리수의 열 단계 차이가 `3` 이하이면 서리 문양을 얻는다. 차이가 크면 검날에 균열이 생긴다.	두 값의 차이를 절댓값으로 계산한다. 차이가 크면 공기로 먼저 식힌 뒤 서리수를 쓴다.	서리 문양을 얻는다. 별철의 조건도 함께 충족할 수 있다.
`9` 용뼈	철 검날, 은도금 검날, 별철 검날 중 완벽하게 가공한 것 하나를 매개체로 사용한다. 용뼈와 매개체를 `100°C`의 물에서 `3초` 동안 처리한다.	검날 상태와 수조 온도를 확인하고, 정제값을 계산해 공정에 전달한다.	정제 용뼈를 손잡이로 쓸 수 있다. 마력 수정과 조합해 고가 장비를 만들 수 있다.

용뼈 정제 예시

- 완벽하게 가공한 검날의 조건은 성형과 냉각 완료, 균열 없음, 최고 마감 등급이다.
- `DragonBone = 9`로 재료 번호를 선언한다. 숫자 함수 `10`을 절댓값 함수로 배정하는 안에서는 `Refinement = 10(DragonBone + 4)`로 정제값 `13`을 계산한다. 에디터 툴팁은 `10 · 절댓값`으로 표시한다.
- `13`은 정제 공정에 전달하는 설정값이다. 공방은 검날, 수조 온도, 설정값을 검사한 뒤 정제 공정을 수행한다.
- 검날의 `열 단계`와 수조의 `온도(°C)`는 별도 값으로 표시한다.

입력 예시	공정 결과
정제값 `13`, 완벽한 검날, 수조 `100°C`	`3초`간 정제해 장착 가능한 용뼈를 얻는다.
정제값 `13`, 완벽한 검날, 수조 `95°C`	시작 전에 멈추고 `수조 온도 95°C / 필요 100°C`를 표시한다.
정제값 `12`, 완벽한 검날, 수조 `100°C`	시작 전에 멈추고 `정제값 12 / 필요 13`을 표시한다.

재료 조합을 발견하는 예시

별철 검날이 열 단계 `7`이고 서리수가 열 단계 `4`이면 두 재료의 조건을 함께 만족한다. 냉각 한 번으로 별무늬와 서리 문양을 모두 얻을 수 있다. 진열대에서 재료별 조건을 읽고, 제작 결과와 판매 영수증에서 어떤 특성이

함께 붙었는지 확인하는 흐름을 제안한다.

9. 확정 예약어와 인게임 적용 프롬프트

기록일 @2026년 11월 8일

상태 · 예약어 대응표 확정, 구현 지시 정리. 보라색 강조를 적용하며, 구체적인 색상 `#C586C0`은 초기 제안입니다. 아래 내용은 구현 요청이며 실제 구현 완료 여부는 별도로 확인해야 합니다.

이전 초안과의 관계 앞 절의 `? 조건:`과 절댓값 함수 `10(...)`은 이전 제안입니다. 이번 적용 기준은 `if → 1?`, `abs → 101()` 입니다.

인게임 숫자 예약어 및 구문 강조 적용 프롬프트

Unity 6 프로젝트 `2026Unity-GameJam`의 인게임 코드 에디터와 도면 실행 시스템에 아래 예약어를 적용해주세요. 현재 프로젝트의 코드와 기획을 기준으로 에디터, 토크나이저, 파서, 실행기 구조를 확인한 뒤 구현해주세요.

예약어 대응표

기존 문법·함수	인게임 표기
`while()`	`888()`
`for()`	`123()`
`break`	`!!`
`continue`	`~~`
`if()`	`1?`
`elif()`	`01?`
`else`	`0?`
`def()`	`333()`
`return`	`<<`
`heat()`	`100()`
`abs()`	`101()`
`len()`	`1111()`
`min()`	`000()`
`max()`	`999()`
`random()`	`66()`
`set()`	`1010()`

괄호, 콜론, 들여쓰기와 인자 배치는 현재 프로젝트의 문법 구조에 맞춰 적용해주세요. 반복문, 조건문, 함수 정의, 함수 호출을 각각 올바른 문법 요소로 처리해주세요.

1. 영문 표기와 숫자·기호 표기를 같은 기능으로 연결해주세요.

두 표기가 동일한 내부 명령 또는 함수 식별자를 사용하도록 구현해주세요.

예를 들어 `abs(-3)`과 `101(-3)`은 같은 절댓값 기능을 호출하며 결과가 모두 `3`이어야 합니다. `heat(Blade)`와 `100(Blade)`도 동일한 작업물의 열을 읽어야 합니다.

실행 로직을 공유하고 매개변수 검사, 반환값, 실행 순서, 비용, 상태 변화, 오류 조건도 같게 적용해주세요. 한 도면에 영문 표기와 숫자·기호 표기가 섞여 있어도 정상적으로 해석되어야 합니다.

미구현 기능은 현재 프로젝트에서 동작 정의를 확인해주세요. 정의가 부족한 부분은 필요한 결정을 구체적으로 보고해주세요.

2. 문맥을 기준으로 예약어를 인식해주세요.

숫자와 기호가 등장한 위치에 따라 의미를 구분해주세요.

- `Value = 100`의 `100`은 숫자 값입니다.
- `100(Blade)`의 `100`은 `heat` 함수입니다.
- `Value = 888`의 `888`은 숫자 값입니다.
- 반복문 위치의 `888(조건)`은 `while`입니다.
- `000(values)`의 `000`은 `min`이며, 원문 표기 `000`을 유지해야 합니다.
- 문자열과 주석 내부의 내용은 예약어로 해석하지 않아야 합니다.
- 일반 변수명에 포함된 영문 문자열을 예약어로 분리하지 않아야 합니다.

정수 변환 전에 원문과 문맥을 확인하여 `000()`과 `01?`를 인식해주세요. `01?`는 선행 0을 유지한 하나의 예약어이며, `0`과 `1?`로 분리하거나 `1?`로 정규화하지 않아야 합니다.

`!!`, `~~`, `<<`, `1?`, `01?`, `0?`는 각각 하나의 복합 토큰으로 처리해주세요. 기존 연산자와 문법의 충돌을 확인하고 토큰 우선순위를 명확히 정해주세요. 기존 완성품 처리 문법 `!Blade`와 반복문 종료 `!!`도 구분해주세요.

3. 예약어를 보라색으로 강조해주세요.

대응표에 있는 영문 예약어·내장 함수명과 숫자·기호 표기를 동일한 보라색으로 표시해주세요.

초기 색상은 `#C586C0`으로 제안합니다. 기존 테마 설정에 통합하거나 Inspector에서 조정할 수 있도록 구성해주세요.

예약어 토큰 범위만 강조해주세요. `100(Blade)`에서는 `100`을 보라색으로 표시하고, 괄호와 변수에는 기존 강조 규칙을 적용해주세요. `1?`, `01?`, `!!` 등은 토큰 전체를 같은 색으로 표시해주세요.

실행 해석과 구문 강조가 같은 토큰 정보를 활용하도록 구성해주세요. 입력 중인 미완성 코드에서도 강조가 갱신되어야 합니다.

입력·삭제·붙여넣기 과정에서 커서, 선택 영역, 스크롤, 줄바꿈, 들여쓰기, 한글 입력 조합이 정상적으로 유지되는지 확인해주세요.

TMP 리치 텍스트를 사용한다면 표시용 태그와 실제 소스를 분리해주세요. `<<`, `<`, `>`가 정확히 표시되고, 저장하거나 복사한 소스에 색상 태그가 포함되지 않아야 합니다.

4. 두 표기의 대응 관계를 툴팁으로 안내해주세요.

예약어 위에 마우스를 올리면 숫자·기호 표기, 영문 이름, 기능 설명을 함께 표시해주세요. 기존 툴팁 시스템이 있다면 활용해주세요.

표시 예시	설명
`888 ↔ while`	조건이 참인 동안 블록을 반복합니다.
`123 ↔ for`	지정된 대상이나 범위를 순회합니다.
`!! ↔ break`	가장 가까운 반복문을 종료합니다.
`~~ ↔ continue`	현재 반복의 남은 부분을 건너뛰고 다음 반복으로 이동합니다.

표시 예시	설명
`1? ↔ if`	조건이 참이면 블록을 실행합니다.
`01? ↔ elif`	같은 조건문에서 앞선 조건들이 모두 거짓이고 현재 조건이 참이면 블록을 실행합니다.
`0? ↔ else`	연결된 앞 조건 분기에서 실행할 블록이 선택되지 않았을 때 실행합니다.
`333 ↔ def`	사용자 함수를 정의합니다.
`<< ↔ return`	현재 함수를 종료하고 호출한 위치로 결과를 돌려줍니다.
`100 ↔ heat`	작업물의 현재 열 단계를 읽습니다.
`101 ↔ abs`	절댓값을 구합니다.
`1111 ↔ len`	원소 개수를 구합니다.
`000 ↔ min`	최솟값을 구합니다.
`999 ↔ max`	최댓값을 구합니다.
`66 ↔ random`	지정한 범위에서 무작위 값을 구합니다.
`1010 ↔ set`	중복 원소를 허용하지 않는 집합을 만듭니다.

인자 형식, 반환값, 난수 범위의 경계 포함 여부 등 세부 안내는 실제 구현과 일치시켜주세요.

5. 향후 코드 표기 번역에 사용할 대응 데이터를 구성해주세요.

다음 정보를 하나의 대응 데이터에서 관리해주세요.

- 내부 기능 식별자
- 영문 표기
- 숫자·기호 표기
- 문법 종류
- 설명 또는 현지화 키

파서, 구문 강조, 툴팁에서 이 정보를 함께 활용하고, 향후 영문 코드와 숫자·기호 코드 사이의 표기 변환에도 사용할 수 있도록 구성해주세요.

변환은 토큰의 의미를 기준으로 처리할 수 있어야 합니다. 함수 호출 `100(Blade)`는 `heat(Blade)`로 변환하지만, 숫자 리터럴 `Target = 100`과 문자열·주석 내용은 그대로 유지하는 방식입니다.

현재 작업 범위는 예약어 별칭, 대응 데이터, 보라색 강조, 툴팁입니다. 표기 번역 화면은 후속 기능으로 준비해주세요.

6. 실제 인게임 실행과 편집 동작을 확인해주세요.

다음 항목을 확인해주세요.

- 영문 도면과 숫자·기호 도면의 실행 결과가 동일한지
- 중첩된 반복문과 함수에서 `break`, `continue`, `return`이 정확한 범위에 작용하는지
- 숫자 리터럴, 문자열, 주석을 예약어로 오인하지 않는지
- `000()`과 `01?`를 포함한 복합 기호가 저장·불러오기 이후에도 유지되는지
- `if → elif → else`와 `1? → 01? → 0?`가 같은 분기를 선택하며, 여러 `elif` 중 처음 참인 분기만 실행되는지
- `elif`와 `01?`가 연결할 `if` 없이 사용되거나 `else` 뒤에 나타나면 문법 오류로 안내하는지
- 구문 강조와 툴팁이 실제 편집 화면에서 정상적으로 작동하는지

- 기존 제작과 완성품 처리 흐름이 정상적으로 실행되는지

완료 후 수정한 파일과 관련 함수, 사용할 수 있는 문법 예시, 인게임 적용 방법, 확인 결과를 보고해주세요. 구현 과정에서 판단한 사항과 추가 결정이 필요한 사항도 함께 알려주세요.